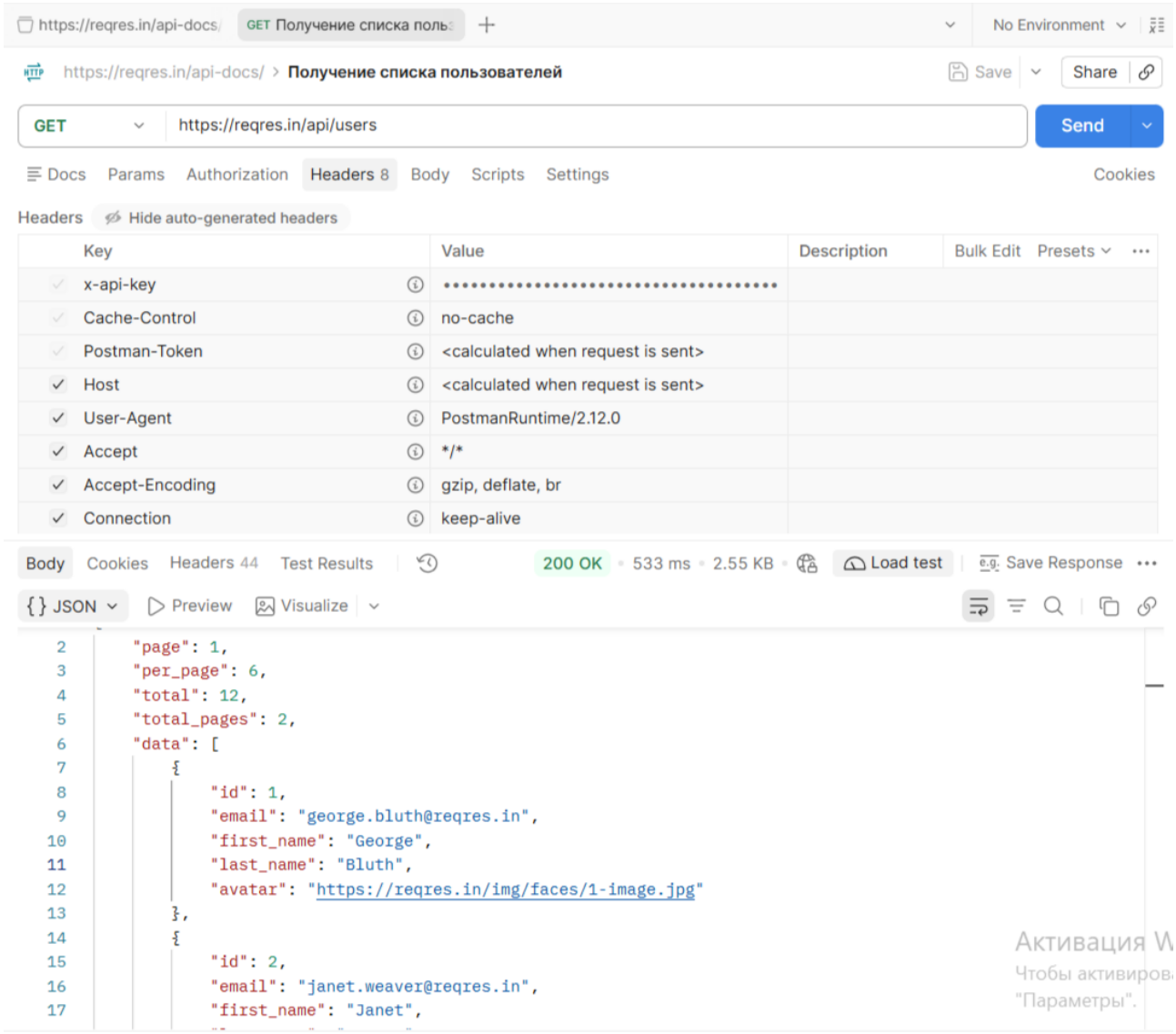
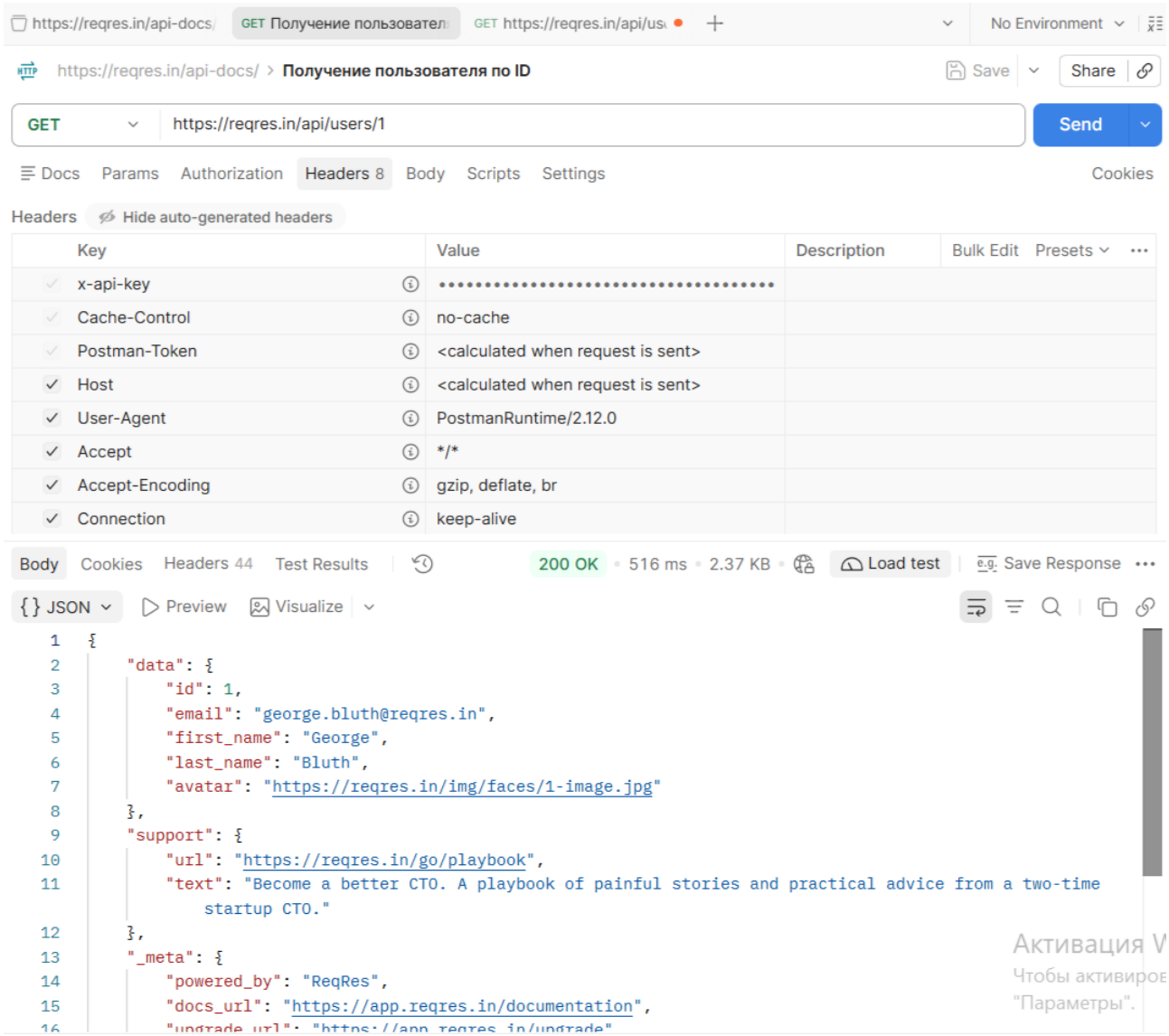
Тестирование REST API

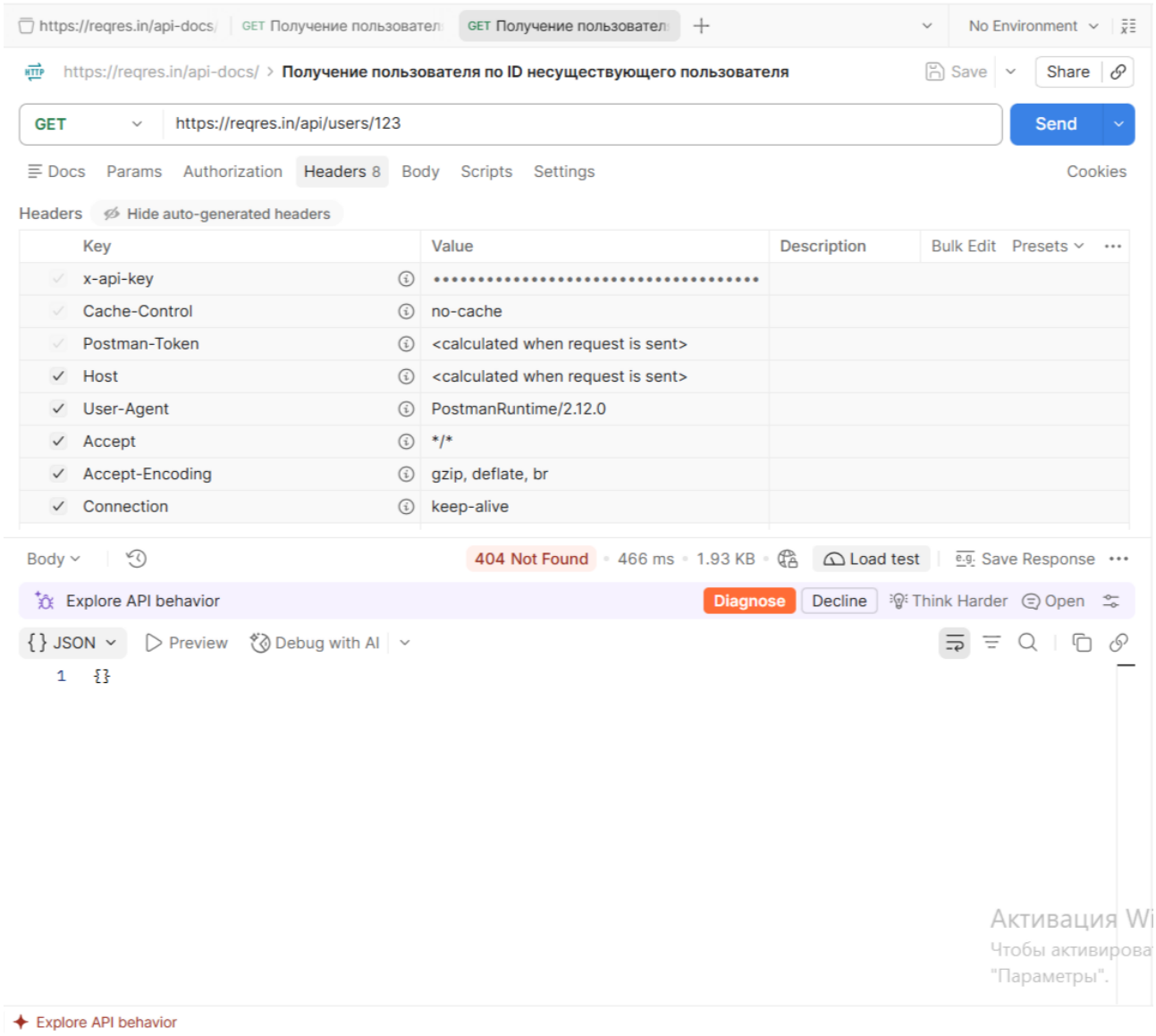
1. Скриншоты выполненных запросов (URL, заголовки, тело, ответ)

GET /api/users— получение списка пользователей

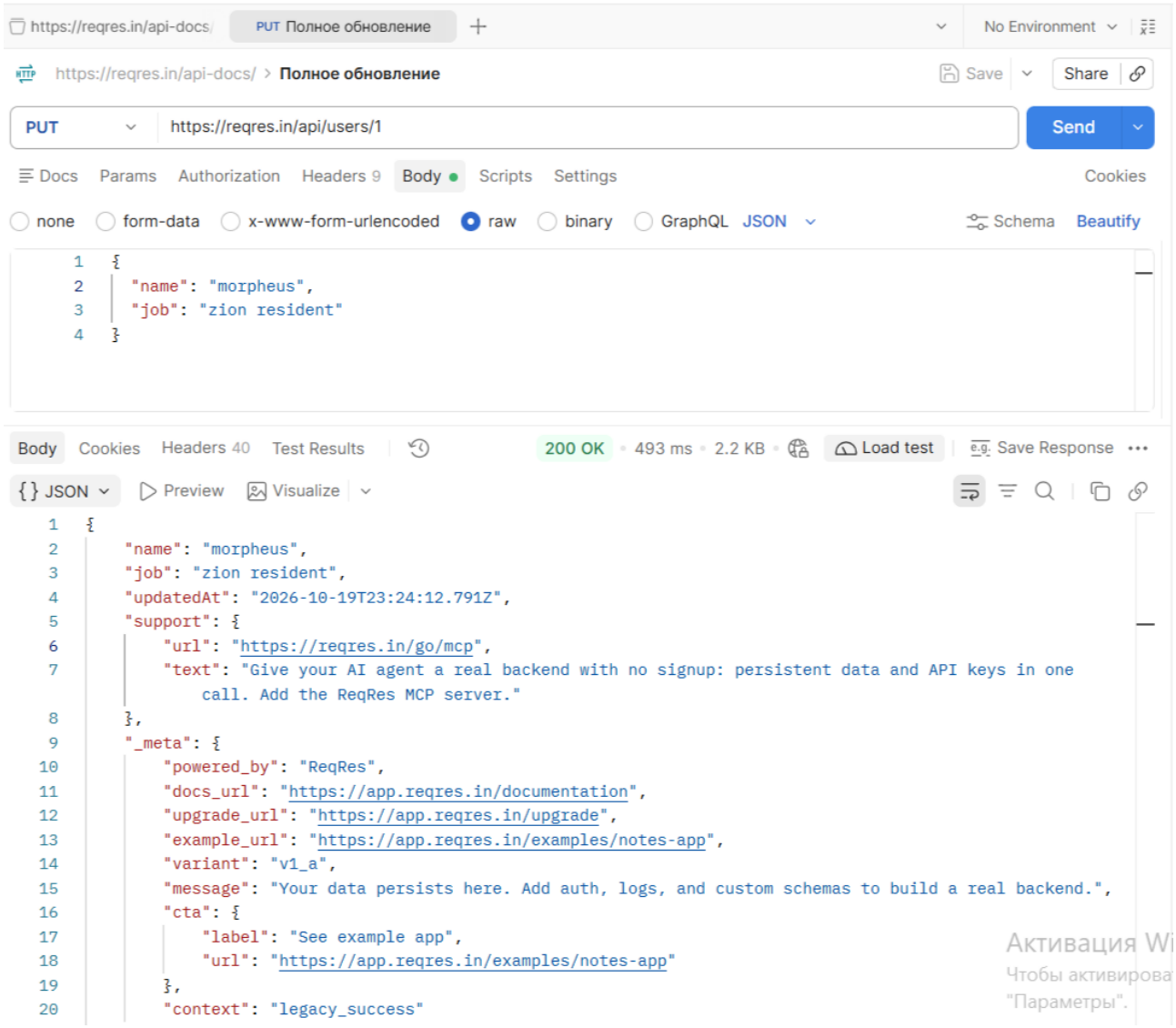


GET /api/users/{id}— получение пользователя по ID

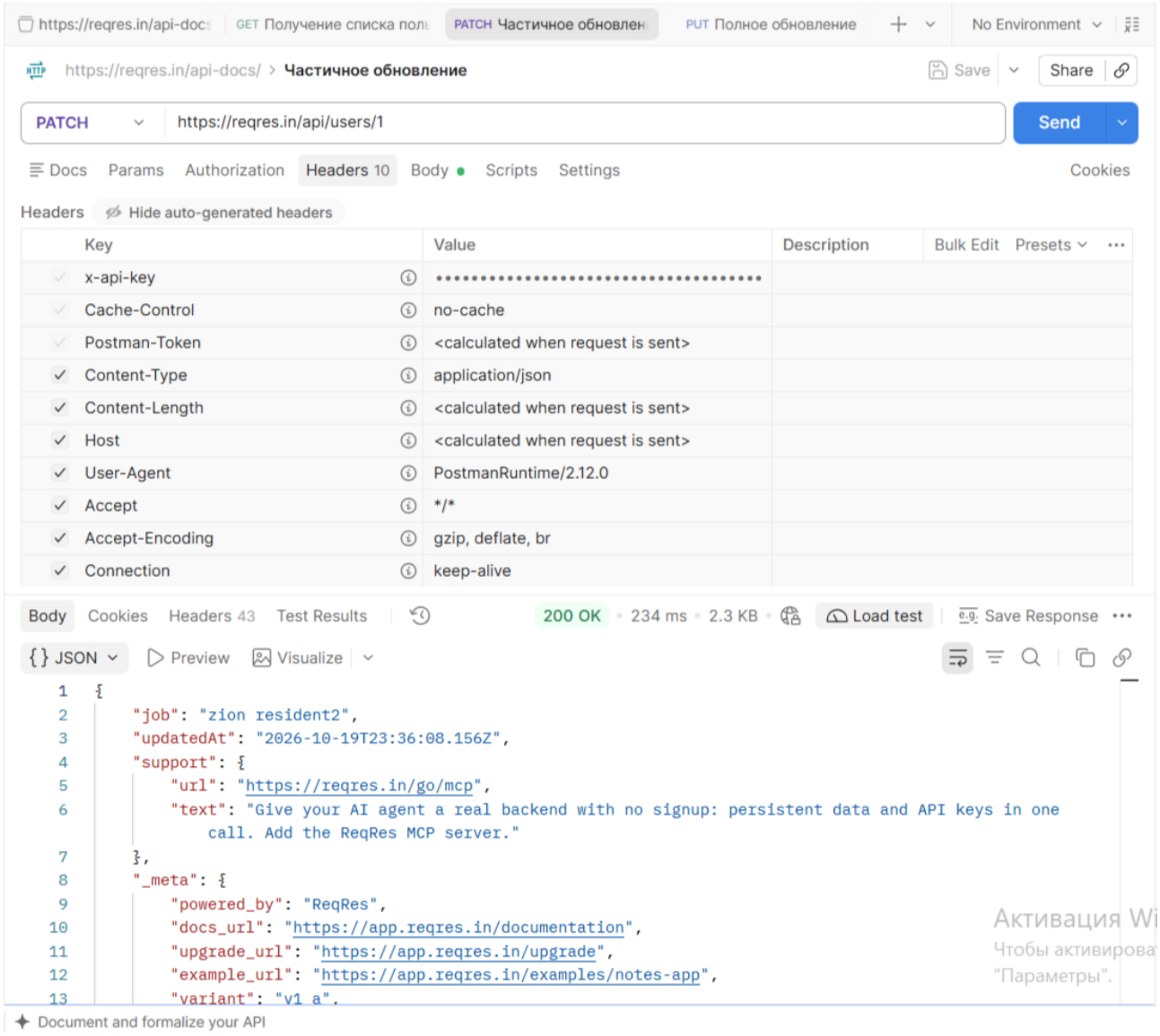


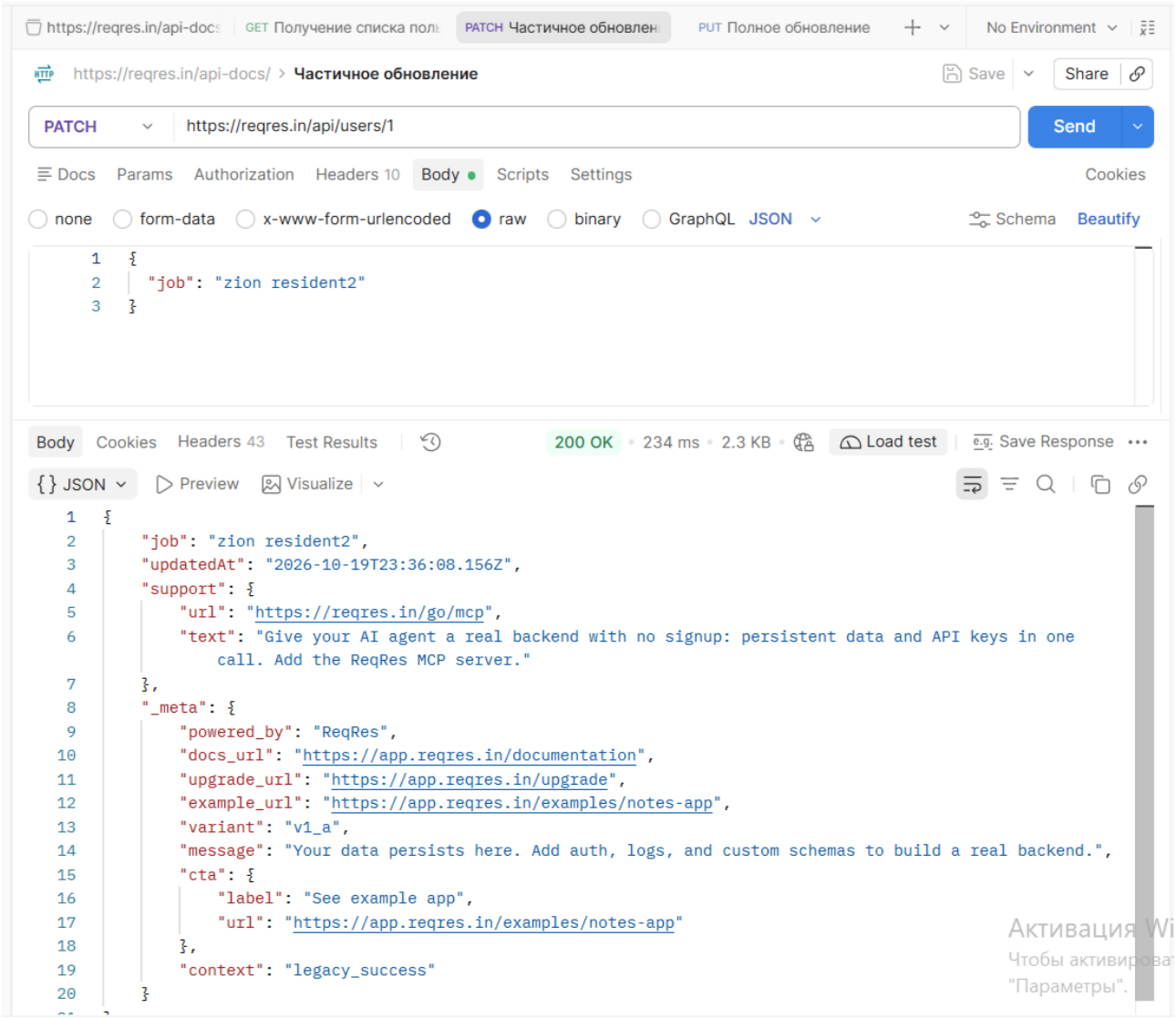

https://reqres.in/api-docs/
GET Получение пользовател
GET Получение пользовател
No Environment
https://reqres.in/api-docs/ > Получение пользователя по ID несуществующего пользователя
Save
Share
GET
https://reqres.in/api/users/123
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Headers
Hide auto-generated headers
Key
Value
Description
Bulk Edit
Presets
x-api-key
Cache-Control
no-cache
Postman-Token
<calculated when request is sent>
Host
<calculated when request is sent>
User-Agent
PostmanRuntime/2.12.0
Accept
/
Accept-Encoding
gzip, deflate, br
Connection
keep-alive
Body
404 Not Found
466 ms
1.93 KB
Load test
Save Response
Explore API behavior
Diagnose
Decline
Think Harder
Open
{} JSON
Preview
Debug with AI
1 {}
Активация Wi
Чтобы активирова
"Параметры".
Explore API behavior

PUT-запрос (полное обновление)

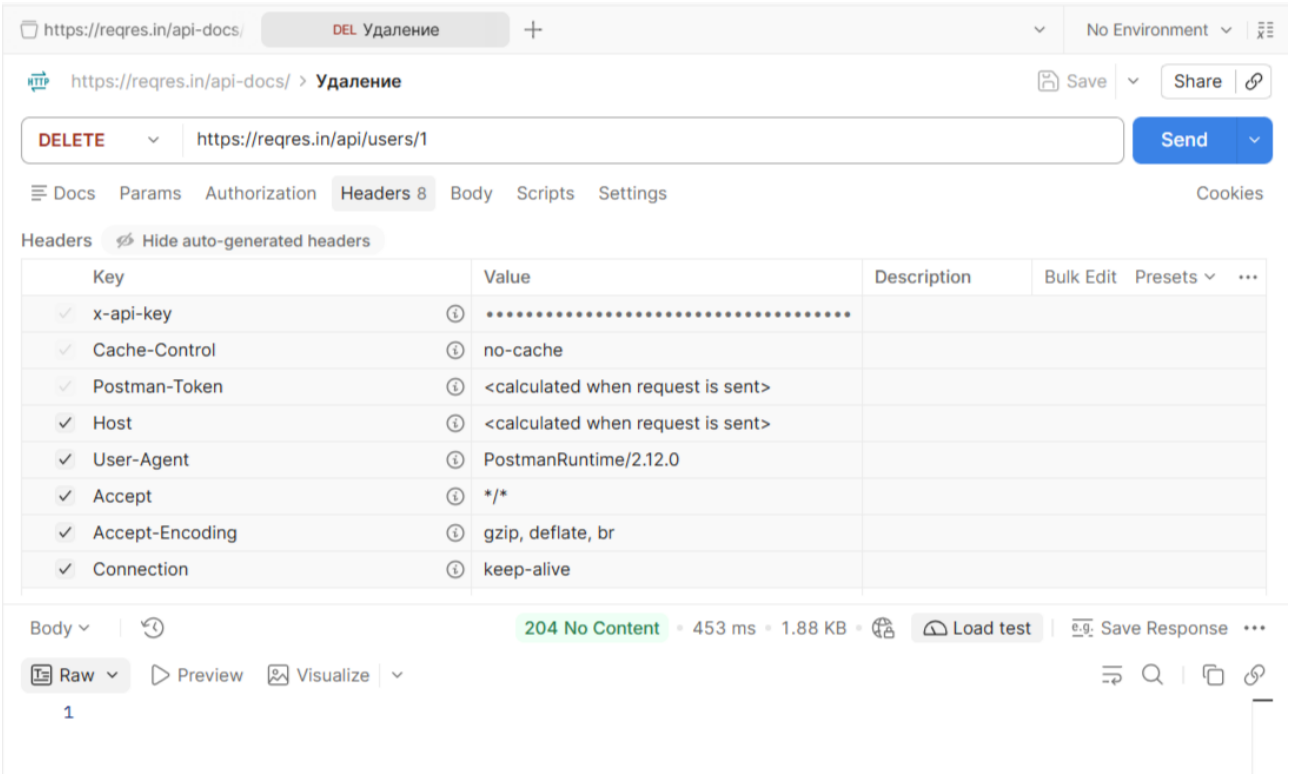


PATCH-запрос (частичное обновление)




https://reqres.in/api-docs
GET Получение списка пол
PATCH Частичное обновлен
PUT Полное обновление
No Environment
https://reqres.in/api-docs/ > Частичное обновление
Save
Share
PATCH
https://reqres.in/api/users/1
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"job": "zion resident2"
}
Body
Cookies
Headers 43
Test Results
200 OK
234 ms
2.3 KB
Load test
Save Response
{ } JSON
Preview
Visualize
{
"job": "zion resident2",
"updatedAt": "2026-10-19T23:36:08.156Z",
"support": {
"url": "https://reqres.in/go/mcp",
"text": "Give your AI agent a real backend with no signup: persistent data and API keys in one call. Add the ReqRes MCP server."
},
"_meta": {
"powered_by": "ReqRes",
"docs_url": "https://app.reqres.in/documentation",
"upgrade_url": "https://app.reqres.in/upgrade",
"example_url": "https://app.reqres.in/examples/notes-app",
"variant": "v1_a",
"message": "Your data persists here. Add auth, logs, and custom schemas to build a real backend.",
"cta": {
"label": "See example app",
"url": "https://app.reqres.in/examples/notes-app"
},
"context": "legacy_success"
}

DELETE-запрос



2. Таблица соответствия HTTP-методов и CRUD

CRUD-операция	HTTP-метод	Что делает	Пример в Postman
Create	POST	Создаёт пользователя	Добавить нового пользователя
Read	GET	Получает данные	Получить список пользователей
Update	PUT	Полностью обновляет данные	Изменить все поля пользователя
Update	PATCH	Частично обновляет данные	Изменить только имя
Delete	DELETE	Удаляет данные	Удалить пользователя

3. Список всех полученных HTTP-статус-кодов с пояснением каждого

- 200 OK — Запрос успешно выполнен.
- 404 Not Found — Запрашиваемый ресурс не найден.
- 204 No Content — Запрос успешно выполнен, но сервер не возвращает содержимое ответа. Например, ресурс успешно удалён.

4. Различия между REST и SOAP на основе выполненных заданий

1. SOAP API — это протокол обмена данными, который использует XML для отправки запросов и получения ответов. В задании был протестирован сервис «Калькулятор» с методами Add, Subtract, Multiply и Divide. Для выполнения запросов использовался WSDL, описывающий доступные методы и их параметры.
2. REST API — это архитектурный стиль взаимодействия с сервером, который использует HTTP-методы для выполнения операций с ресурсами. В задании тестировалось управление пользователями с помощью методов GET, POST, PUT, PATCH и DELETE. Данные передавались и получались в формате JSON.

Основные различия:

Критерий	**SOAP**	**REST**
Тип	Протокол	Архитектурный стиль
Формат данных	XML	Чаще всего JSON
Описание сервиса	WSDL	Обычно документация API
Методы взаимодействия	Например, Add, Subtract, Multiply, Divide	GET, POST, PUT, PATCH, DELETE
Назначение в задании	Выполнение математических операций	Получение, создание, изменение и удаление пользователей
Обработка ошибок	Возвращаются SOAP Fault и HTTP-ошибки	Используются HTTP-статус-коды, например 200, 204, 404, 500